

STRING

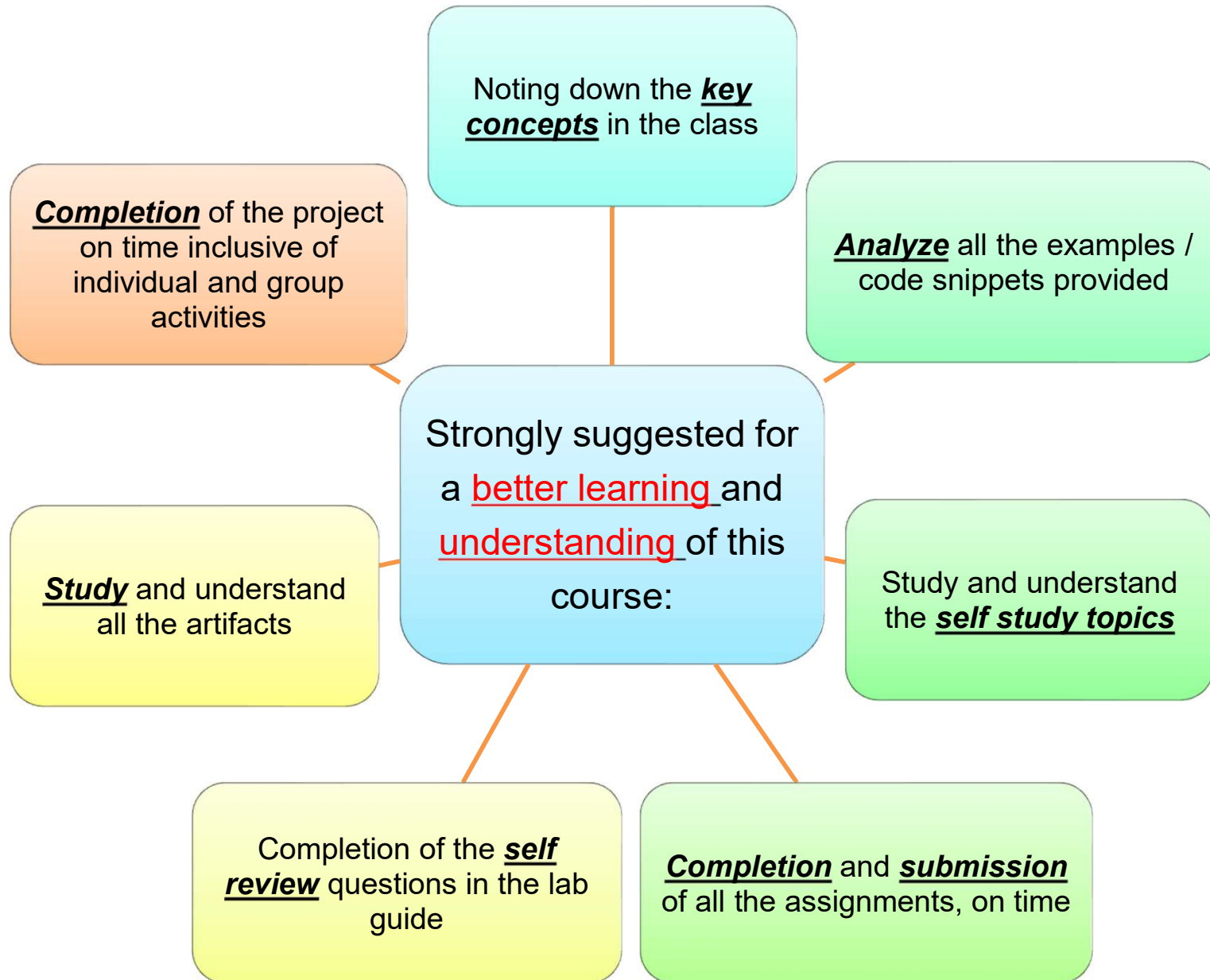
Instructor:



Table of contents

- ◇ **String**
- ◇ **StringBuffer**
- ◇ **StringBuilder**

Learning Approach

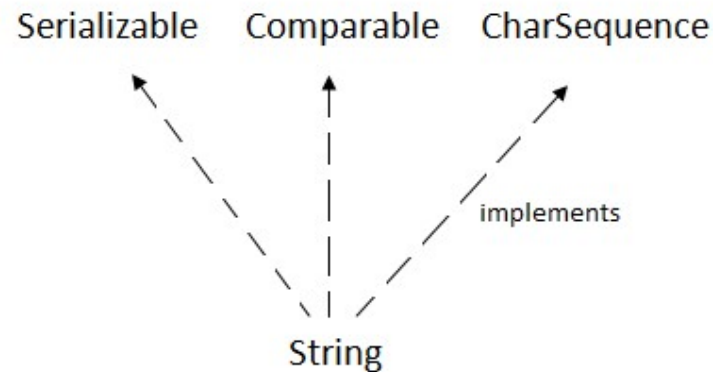


Section 1

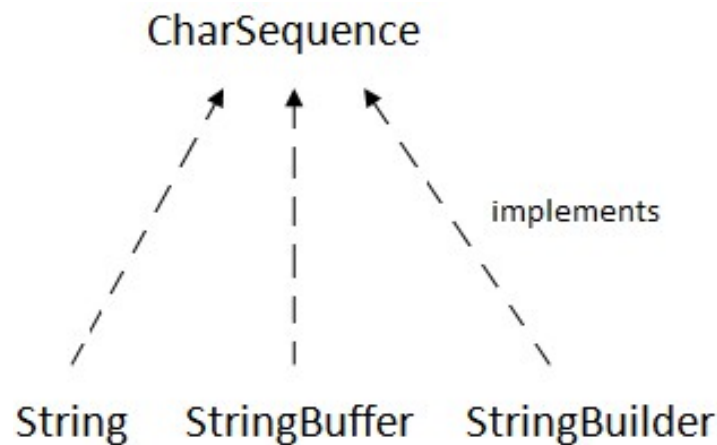
STRING CLASS

- ❖ **String** is a sequence of characters, for e.g. “Hello” is a string of 5 characters.
- ❖ In java, string is an **immutable object** which means it is **constant** and can **cannot be changed** once it has been created.
- ❖ **Creating a String**
 - ✓ There are two ways to create a String in Java
 - String **literal**
 - Using **new** keyword

- ❖ The **java.lang.String** class implements **Serializable**, **Comparable** and **CharSequence** interfaces.



- ❖ The **CharSequence** interface is used to represent the sequence of characters. **String**, **StringBuffer** and **StringBuilder** classes implement it.



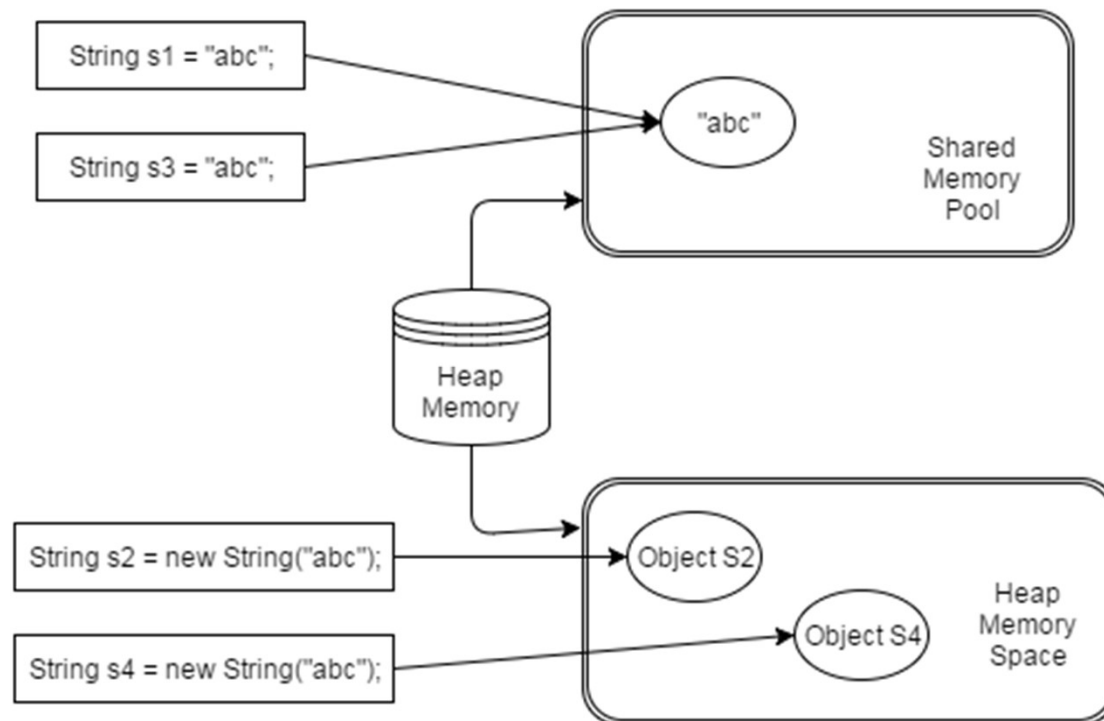
String literal

- ❖ In java, Strings can be created like this: Assigning a String literal to a String instance.

- ❖ Example:

```
String s1= "abc";
```

```
String s3= "abc";
```



String object

- ❖ Using `new` keyword
- ❖ The compiler would **create two different object** in memory having the **same string**.
- ❖ **Example:**

```
public class StringSample {  
  
    public static void main(String[] args) {  
        // creating a string by java string literal  
        String s1 = "FPT";  
        String s3 = "FPT";  
  
        char arrch[] = { 'h', 'e', 'l', 'l', 'o' };  
        // converting char array arrch[] to string str2  
        String s2= new String(arrch);  
  
        // creating another java string str3 by using new keyword  
        String s4 = new String("hello");  
  
        // Displaying all the three strings  
        System.out.println(s1.equals(s3));  
        System.out.println(s2.equals(s4));  
  
        System.out.println(s1 == s3);  
        System.out.println(s2 == s4);  
    }  
}
```

Output:

```
true  
true  
true  
false
```


❖ Methods:

- ✓ **char charAt(int index)**: It returns the character at the specified index. Specified index value should be between 0 to length() -1 both inclusive. It throws IndexOutOfBoundsException if $\text{index} < 0 || \geq \text{length of String}$.
- ✓ **boolean equals(Object obj)**: Compares the string with the specified string and returns true if both matches else false.
- ✓ **int compareTo(String string)**: This method compares the two strings based on the Unicode value of each character in the strings.
- ✓ **boolean startsWith(String prefix)**: It tests whether the string is having specified prefix, if yes then it returns true else false.
- ✓ **boolean endsWith(String suffix)**: Checks whether the string ends with the specified suffix.
- ✓ **int hashCode()**: It returns the hash code of the string.
- ✓ **int indexOf(int ch)**: Returns the index of first occurrence of the specified character ch in the string.
- ✓ **boolean contains(CharSequence s)**: It checks whether the string contains the specified sequence of char values. If yes then it returns true else false. It throws NullPointerException of 's' is null.

- ✓ **String concat(String str)**: Concatenates the specified string “str” at the end of the string.
- ✓ **String trim()**: Returns the substring after omitting leading and trailing white spaces from the original string.
- ✓ **byte[] getBytes()**: This method is similar to the above method it just uses the default charset encoding for converting the string into sequence of bytes.
- ✓ **int length()**: It returns the length of a String.
- ✓ **boolean matches(String regex)**: It checks whether the String is matching with the specified regular expression regex.
- ✓ **static String valueOf()**: This method returns a string representation of passed arguments such as int, long, float, double, char and char array.
- ✓ **char[] toCharArray()**: Converts the string to a character array.
- ✓ **String[] split(String regex)**: Same as split(String regex, int limit) method however it does not have any threshold limit.

❖ Convert a digit sequence to number

```
// Each class in right hand side is called wrapper
// class of the corresponding primitive type
byte b = Byte.parseByte("128");
                                // NumberFormatException

short s = Short.parseShort("32767");
int x = Integer.parseInt("2");
int y = Integer.parseInt("2.5");
                                // NumberFormatException

int z = Integer.parseInt("a");
                                // NumberFormatException

long l = Long.parseLong("15");
float f = Float.parseFloat("1.1");
double d = Double.parseDouble("2.5");
```

Immutable String

- ❖ In java, **string objects are immutable**. Immutable simply means unmodifiable or unchangeable.
- ❖ **Example:**

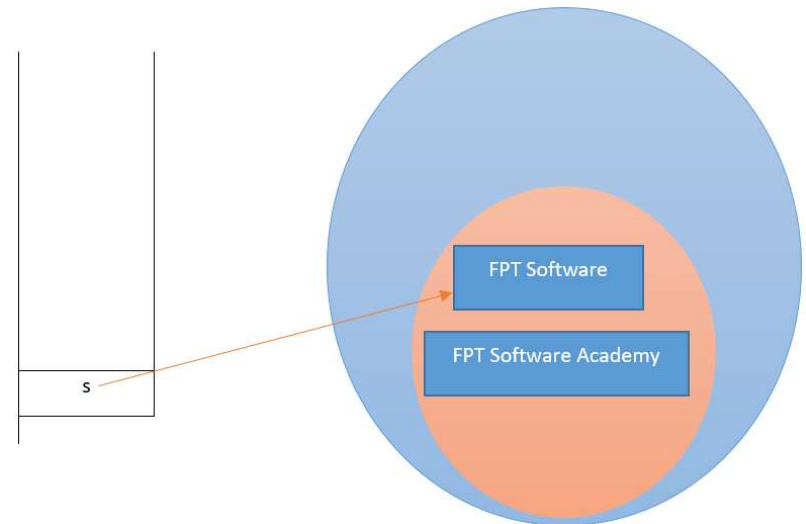
```
public class TestImmutableString {  
  
    public static void main(String[] args) {  
        String s = "FPT Software";  
        s.concat(" Academy"); // concat() method appends the string at the end  
        System.out.println(s); // will print FPT Software because strings are  
                               // immutable objects  
    }  
}
```

- ❖ **Output:**

FPT Software

Solution:

```
s = s.concat(" Academy");
```



String Comparison

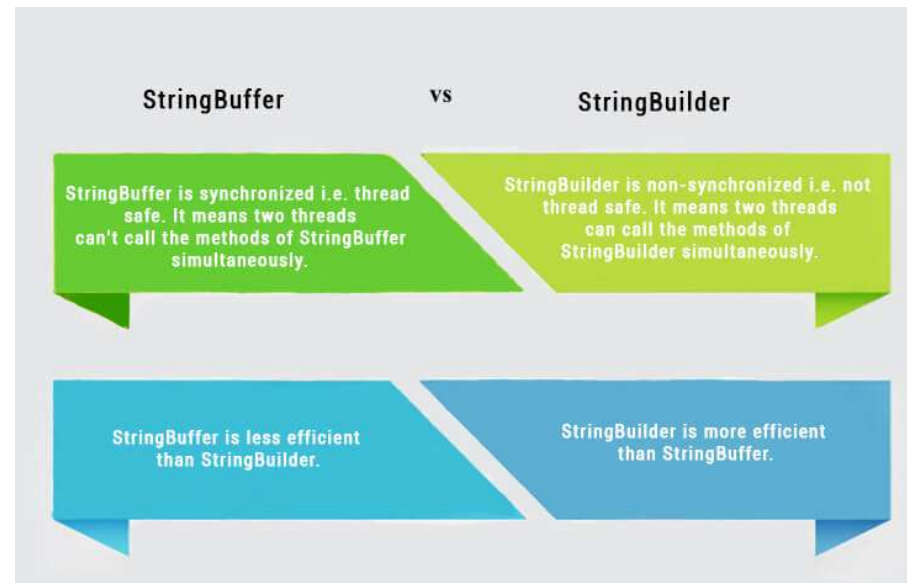
- ❖ We can compare string in java on the basis of content and reference.
- ❖ It is used in **authentication** (by equals() method), **sorting** (by compareTo() method), **reference matching** (by == operator) etc.
- ❖ There are three ways to compare string in java:
 - ✓ In **authentication**: by equals()/equalsIgnoreCase() method
 - ✓ In **sorting**: by == operator
 - ✓ **Reference matching**: by compareTo() method. This method compares values **lexicographically** (từ vựng) and returns an integer value that describes if first string is *less than, equal to* or *greater* than second string.

Section 2

STRINGBUILDER & STRINGBUFFER

StringBuilder and StringBuffer class

- ❖ The **StringBuffer** and **StringBuilder** classes: to make a lot of modifications to Strings of characters.
- ❖ The **StringBuilder** class was introduced as of Java 5 and the main difference between the **StringBuffer** and **StringBuilder** is that **StringBuilders** methods are *not thread safe* (not Synchronised).
- ❖ It is recommended to use **StringBuilder** whenever possible because it is faster than **StringBuffer**. However if thread safety is necessary the best option is **StringBuffer** objects.



String/StringBuilder/StringBuffer

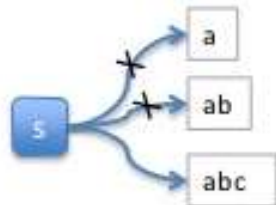
- ❖ String is **immutable**, if you try to alter their values, another object gets created,
- ❖ **StringBuffer** and **StringBuilder** are **mutable** so they can change their values.

String

Immutable

Every time you alter String values, it will allocate another exact amount of space in the heap. The previous value in the memory will be garbage-collected later.

```
String s = "a";  
s += "b";  
s += "c";
```



StringBuffer

Mutable

When created it reserves a certain amount of space in the heap, which can be larger than the value. Within that space, values can be modified without additional memory use. When the value requires more space, the space will automatically grow larger.

```
StringBuffer s = new StringBuffer("a");  
s.append("b");  
s.append("c");
```



Methods of StringBuffer class

Method	Description
public synchronized StringBuffer append(String s)	Is used to append the specified string with this string. The append() method is overloaded like: • append(char), • append(boolean), • append(int), • append(float), • append(double) etc.
public synchronized StringBuffer insert(int offset, String s)	Is used to insert the specified string with this string at the specified position. The insert() method is overloaded like: • insert(int, char), insert(int, boolean), • insert(int, int), insert(int, float), insert(int, double) etc.
public synchronized StringBuffer replace(int startIndex, int endIndex, String str)	Is used to replace the string from specified startIndex and endIndex.
public synchronized StringBuffer delete(int startIndex, int endIndex)	Is used to delete the string from specified startIndex and endIndex.
public synchronized StringBuffer reverse()	Is used to reverse the string.

Methods of StringBuffer class

Method	Description
public int capacity ()	Is used to return the current capacity.
public void ensureCapacity (int minimumCapacity)	Is used to ensure the capacity at least equal to the given minimum.
public char charAt (int index)	Is used to return the character at the specified position.
public int length ()	Is used to return the length of the string i.e. total number of characters.
public String substring (int beginIndex)	Is used to return the substring from the specified beginIndex .
public String substring (int beginIndex, int endIndex)	Is used to return the substring from the specified beginIndex and endIndex .

StringBuilder class

- ❖ The Java **StringBuilder** class is **same as StringBuffer** class except that it is non-synchronized.

No.	StringBuffer	StringBuilder
1)	StringBuffer is <i>synchronized</i> i.e. thread safe. It means two threads can't call the methods of StringBuffer simultaneously.	StringBuilder is <i>non-synchronized</i> i.e. not thread safe. It means two threads can call the methods of StringBuilder simultaneously.
2)	StringBuffer is <i>less efficient</i> than StringBuffer.	StringBuilder is <i>more efficient</i> than StringBuffer.

❖ Example:

```
public class ConcatTest {  
    public static void main(String[] args) {  
        long startTime = System.currentTimeMillis();  
  
        StringBuffer sb = new StringBuffer("Java");  
        for (int i = 0; i < 1000000; i++) {  
            sb.append(" Learning");  
        }  
  
        System.out.println("Time taken by StringBuffer: "  
            + (System.currentTimeMillis() - startTime) + "ms");  
    }  
}
```

```
        startTime = System.currentTimeMillis();  
  
        StringBuilder sb2 = new StringBuilder("Java");  
        for (int i = 0; i < 1000000; i++) {  
            sb2.append(" Learning");  
        }  
  
        System.out.println("Time taken by StringBuilder: "  
            + (System.currentTimeMillis() - startTime) + "ms");  
    }  
}
```

❖ Output:

Time taken by StringBuffer: 51ms

Time taken by StringBuilder: 26ms

StringBuilder/StringBuffer examples

```
StringBuilder sb = new StringBuilder("abc");  
✓ sb.append(" def"); // "abc def"  
✓ char letter = str.charAt(2); // "b"  
✓ char ch[] = new char[3];  
  str.getChars(1, 3, ch, 0); // Bây giờ biến "ch" chứa "abc"  
✓ sb.delete(3, 5); // "abcef"  
✓ sb.deleteCharAt(4); // "abce"  
✓ sb.insert(3, " d"); // "abc de"  
✓ sb.replace(2, 4, " ghi"); // "ab ghide"  
✓ sb.reverse(); // "edihg ba"  
✓ sb.setCharAt(5, 'j'); // "edihgjba"
```

❖ **StringTokenizer** can be used to parse a line into words

- ✓ `import java.util.*`
- ✓ some of its useful methods are shown in the text
 - e.g. test if there are more tokens
- ✓ you can specify *delimiters* (the character or characters that separate words)
 - the default delimiters are "white space" (space, tab, and newline)

Example: StringTokenizer

- ❖ Display the words separated by any of the following characters: space, new line (\n), period (.) or comma (,).

```
String inputLine = keyboard.nextLine();  
StringTokenizer wordFinder =  
    new StringTokenizer(inputLine, " \n.,");  
//the second argument is a string of the 4delimiters  
while (wordFinder.hasMoreTokens())  
{  
    System.out.println(wordFinder.nextToken());  
}
```

Entering "Question, 2b.or !tooBee."
gives this output:

```
Question  
2b  
or  
!tooBee
```

- ❖ **String**
- ❖ **StringBuffer**
- ❖ **StringBuilder**

Thank you

